

INTELIGENCIA ARTIFICIAL

PEC1 – 2010_1 Prueba de Evaluación Continua

- En caso de dudas o necesitar aclaraciones sobre el enunciado, dirigirse al consultor responsable del aula.
- Es necesario entregar la solución en un archivo PDF usando la plantilla que se adjunta con este enunciado. Adjuntar el fichero en un mensaje en el apartado de **Entrega y registro de EC (REC)** de vuestra aula.
- El nombre del fichero debe ser *ApellidosNombre_IA_PEC1* con la extensión .pdf (PDF).
- En caso que la entrega sea muy grande, podéis entregar la PEC comprimida en un fichero ZIP.
- La fecha límite de entrega es el: **18 de Octubre** (a las 24 horas).
- **Debéis razonar la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.**

Enunciado

En un tablero de 3x3 hay una pieza por casilla, cada una de un color. Hay 4 posibles colores: azul (b), verde (g), rojo (r) y amarillo (y). Los colores se ordenan de más débil a más fuerte del siguiente modo: azul es más débil que verde, verde es más débil que rojo, rojo es más débil que amarillo y el amarillo es el más fuerte. Si denominamos '<' a la relación “ser más débil que” podríamos decir que: azul < verde < rojo < amarillo, o también, en la versión abreviada, $b < g < r < y$ (notemos que la relación '<' es transitiva).

El tablero inicialmente tiene un color en cada casilla (inicialmente podemos tener cualquier color en cualquier casilla, cualquier color puede aparecer cualquier número de veces), por ejemplo:

b	g	r
g	r	y
y	r	b

A partir de esta situación podemos definir un juego para dos jugadores, que juegan por turnos, de manera alternada.

Cada jugador puede escoger una casilla con color y una dirección (horizontal - izquierda/derecha - o vertical - arriba/abajo -, siempre que sea posible).

Entonces el color se moverá en la dirección indicada, “comiéndose” a los colores más débiles, hasta que se encuentra en una de las siguientes situaciones:

- encuentra un color más fuerte (en ese caso, este color se lo “come” a él)
 - encuentra un color igual (en ese caso los dos desaparecen)
 - se encuentra sin más casillas que recorrer (el tablero se ha acabado en la dirección escogida)
 - se encuentra con una casilla vecina, en la dirección del desplazamiento, vacía (por lo tanto un color tan solo puede moverse en una dirección si en esta dirección hay algún color en la casilla vecina).
- Gana el jugador que hace el último movimiento (o, lo que es lo mismo, pierde el que se queda sin poder mover ningún color).

Por ejemplo: Si partimos del tablero del ejemplo anterior y escogemos el color amarillo de la casilla (2,3) (consideremos que la casilla (1,1) es la de arriba a la izquierda) en dirección horizontal hacia la izquierda, el tablero quedará:

b	g	r
y		
y	r	b

ya que y es el color más fuerte y se ha “comido” a todos los colores que se ha encontrado hasta el final del recorrido. Con este tablero podemos ilustrar diversas posibilidades:

a) No podemos escoger r de (1,3) en dirección vertical hacia abajo, ni g de (1,2) en dirección vertical hacia abajo, ni y de (2,1) en dirección horizontal derecha, ni r de (3,2) en dirección vertical hacia arriba, ni b de (3,3) en dirección vertical hacia arriba porque los vecinos en las direcciones indicadas son casillas vacías. Recordemos que es necesario un color en una casilla vecina para cualquier movimiento en la dirección escogida y que una casilla vacía detiene el movimiento de un color.

b) Escogiendo y de (3,1) en dirección vertical hacia arriba el tablero queda:

b	g	r
	r	b

ya que y se encuentra con otro y y, siguiendo las instrucciones, los dos desaparecen.

c) Escogiendo r de (3,2) en dirección horizontal izquierda termina así:

b	g	r
y		
y		b

ya que $r < y$, y en cambio si escogemos la dirección horizontal derecha, tendremos

b	g	r
y		
y		r

ya que $b < r$.

Hay otras posibilidades, pero esperamos que con estos ejemplos la mecánica del juego haya quedado clara (podéis visitar <http://www.cut-the-knot.org/Games/cball.shtml> para jugar a este juego y así entenderlo mejor; es el mismo juego pero jugado con frutas, que representan los colores)

Para las siguientes preguntas partiremos del estado inicial:

b	g	r
	r	b

Se pide formalizar este problema y responder las preguntas enunciadas en los siguientes apartados:

1. ¿Qué información tendrá cada estado? ¿Cuántos estados posibles habrá en el grafo de estados? ¿Son todos los estados accesibles desde el estado inicial dado?

En cada estado tan solo es necesario saber qué color hay en cada casilla, o si una casilla está vacía. Podemos representar un tablero como un *string* de caracteres, representando con una X el espacio vacío. Así pues, el primer tablero del enunciado sería 'bgrgryrb', el tablero después de mover y de (2,3) hacia la izquierda sería 'bgryXXyrb'.

El número de estados posible es sencillamente $5^9 = 1.953.125$, ya que cada combinación de colores y espacios vacíos es un estado válido del juego. En cambio, el número de estados iniciales es tan solo $4^9 = 262.144$, ya que no hay espacios vacíos en el estado inicial.

Obviamente no todos los estados son accesibles desde estado inicial. Solo aquellos resultantes de las operaciones mencionadas en el enunciado a partir del estado inicial (basta pensar que no pueden aparecer colores nuevos, siguiendo las reglas del juego).

2. ¿Cuántos operadores tendremos? ¿Cuáles son?

Tendremos un operador por cada movimiento posible de cualquier color a una casilla determinada. La aplicabilidad o no del operador dependerá de la configuración específica del tablero. La idea sería tener un operador para mover el color de la casilla X en una dirección Y. Como hay 9 casillas y 4 direcciones tendríamos 36 operadores. Sin embargo, podemos desestimar bastantes operadores ya que no tienen sentido: el operador “mover el color de la casilla (1,1) en horizontal izquierda” no existe ya que no hay nada a la izquierda de la casilla (1,1). Así pues, si omitimos los operadores que sabemos que no tienen sentido, nos quedamos con **24** operadores.

3. Dar la definición del estado inicial (según vuestra representación) y describir cómo identificar el estado objetivo.

El estado inicial será 'bgrXXXXrb', siguiendo la representación mencionada anteriormente. Reconocemos el estado objetivo ya que, siguiendo las reglas del juego, no podemos mover ningún color.

4. Aplicar los algoritmos de búsqueda en anchura y profundidad para encontrar una solución del problema. Notemos que cada nivel del árbol de búsqueda corresponde a jugadores que van alternando su turno, pero esto ahora no nos importa. Para *cada desarrollo* del árbol de búsqueda, dar una representación gráfica y responder las siguientes preguntas:

[Importante: Notemos que, tal como está explicado el tema de búsqueda en los materiales de la asignatura, el estado final se identifica *cuando es escogido para generar los sucesores*, y no cuando se incluye en la lista de pendientes.]

a) ¿Cuál de estas 6 opciones habéis escogido para tratar los nodos repetidos?

A: No se han tratado de manera especial.

B: Después de generar los sucesores de un nodo, no se incluyen en la lista de pendientes los nodos que ya han sido tratados.

C: Antes de generar los sucesores de un nodo, compruebo si ya se ha tratado aquel nodo.

D: Después de generar los sucesores de un nodo, no se incluyen en la lista de pendientes los nodos que ya han sido tratados o los que ya están en la lista de pendientes.

E: B+C.

F: Otros tratamientos (detallar).

b) ¿En qué orden habéis aplicado los operadores sobre cada nodo?

En ambos casos (anchura y profundidad) primero hemos intentado mover los colores de las casillas empezando por la (1,1), y luego, si podemos elegir dirección, primero horizontal izquierda y después horizontal derecha (el vertical no importa porque no aparece en el caso concreto que estamos tratando).

c) ¿Cuál es la solución que ha encontrado? ¿Podéis estar seguros de que es la más corta posible?

Profundidad: 'XxrXXXXXr'

Anchura: 'gXrXXXXrX'

Por definición, la solución encontrada en anchura es la más cercana al estado inicial, en otras palabras, la más corta.

d) ¿Cuántos nodos ha generado?

Profundidad: 13

Anchura: 33

e) ¿Cuál es la cantidad de memoria mayor que habéis necesitado para guardar los nodos pendientes de expansión? (Expresada en número de nodos) ¿Cuántos nodos había pendientes de ser tratados en el momento de encontrar el estado final?

Ambas preguntas tienen la misma respuesta, ya que el máximo de nodos guardados aparece al final.

Profundidad: 9

Anchura: 22

f) ¿Cuál ha sido la profundidad máxima a la que ha tenido que llegar en cada expansión?

Profundidad: 3

Anchura: 2

Búsqueda en profundidad:

Nodo a expandir	Nodos expandidos	Nodos por expandir
bgrXXXXrb	-	XgrXXXXrb gXrXXXXrb bXrXXXXrb rXXXXXXrb bgrXXXXXr bgrXXXXrX
XgrXXXXrb	bgrXXXXrb	XXrXXXXrb XrXXXXXrb XgrXXXXrX XgrXXXXXr gXrXXXXrb bXrXXXXrb rXXXXXXrb bgrXXXXXr bgrXXXXrX
XXrXXXXrb	bgrXXXXrb XgrXXXXrb	XXrXXXXXr XXrXXXXrX XrXXXXXrb XgrXXXXrX XgrXXXXXr gXrXXXXrb bXrXXXXrb rXXXXXXrb bgrXXXXXr bgrXXXXrX
XXrXXXXXr	bgrXXXXrb XgrXXXXrb XXrXXXXrb	XXrXXXXrX XrXXXXXrb XgrXXXXrX XgrXXXXXr gXrXXXXrb bXrXXXXrb rXXXXXXrb bgrXXXXXr bgrXXXXrX

El proceso termina cuando XXrXXXXXr no puede expandirse. Es un estado objetivo.

Búsqueda en anchura:

Nodo a expandir	Nodos expandidos	Nodos por expandir
bgrXXXXrb	-	XgrXXXXrb gXrXXXXrb bXrXXXXrb rXXXXXXrb bgrXXXXXr bgrXXXXrX
XgrXXXXrb	bgrXXXXrb	gXrXXXXrb bXrXXXXrb rXXXXXXrb bgrXXXXXr bgrXXXXrX XXrXXXXrb XrXXXXXrb XgrXXXXrX XgrXXXXXr
gXrXXXXrb	bgrXXXXrb XgrXXXXrb	bXrXXXXrb rXXXXXXrb bgrXXXXXr bgrXXXXrX XXrXXXXrb XrXXXXXrb XgrXXXXrX XgrXXXXXr gXrXXXXrX gXrXXXXXr
bXrXXXXrb	bgrXXXXrb XgrXXXXrb gXrXXXXrb	rXXXXXXrb bgrXXXXXr bgrXXXXrX XXrXXXXrb XrXXXXXrb XgrXXXXrX XgrXXXXXr gXrXXXXrX gXrXXXXXr bXrXXXXrX bXrXXXXXr
rXXXXXXrb	bgrXXXXrb XgrXXXXrb gXrXXXXrb bXrXXXXrb	bgrXXXXXr bgrXXXXrX XXrXXXXrb XrXXXXXrb XgrXXXXrX XgrXXXXXr gXrXXXXrX gXrXXXXXr bXrXXXXrX bXrXXXXXr

		bXrXXXXXr rXXXXXXXr rXXXXXXrX
bgrXXXXXr	bgrXXXXrb XgrXXXXrb gXrXXXXrb bXrXXXXrb rXXXXXXrb	bgrXXXXrX XXrXXXXrb XrXXXXXrb XgrXXXXrX XgrXXXXrX gXrXXXXrX gXrXXXXrX bXrXXXXrX bXrXXXXrX rXXXXXXXr rXXXXXXrX XgrXXXXrX gXrXXXXrX bXrXXXXrX rXXXXXXXr
bgrXXXXrX	bgrXXXXrb XgrXXXXrb gXrXXXXrb bXrXXXXrb rXXXXXXrb bgrXXXXrX	XXrXXXXrb XrXXXXXrb XgrXXXXrX XgrXXXXrX gXrXXXXrX gXrXXXXrX bXrXXXXrX bXrXXXXrX rXXXXXXXr rXXXXXXrX XgrXXXXrX gXrXXXXrX bXrXXXXrX rXXXXXXXr XgrXXXXrX gXrXXXXrX bXrXXXXrX rXXXXXXrX
XXrXXXXrb	bgrXXXXrb XgrXXXXrb gXrXXXXrb bXrXXXXrb rXXXXXXrb bgrXXXXrX bgrXXXXrX	XrXXXXXrb XgrXXXXrX XgrXXXXrX gXrXXXXrX gXrXXXXrX bXrXXXXrX bXrXXXXrX rXXXXXXXr rXXXXXXrX XgrXXXXrX gXrXXXXrX bXrXXXXrX rXXXXXXXr

		XgrXXXXrX gXrXXXXrX bXrXXXXrX rXXXXXXrX XXrXXXXrX XXrXXXXrX
XrXXXXXrb	bgrXXXXrb XgrXXXXrb gXrXXXXrb bXrXXXXrb rXXXXXXrb bgrXXXXXr bgrXXXXrX XXrXXXXrb	XgrXXXXrX XgrXXXXXr gXrXXXXrX gXrXXXXXr bXrXXXXrX bXrXXXXXr rXXXXXXXr rXXXXXXrX XgrXXXXXr gXrXXXXXr bXrXXXXXr rXXXXXXXr XgrXXXXrX gXrXXXXrX bXrXXXXrX rXXXXXXrX XXrXXXXXr XXrXXXXrX XrXXXXXXr XrXXXXXrX
XgrXXXXrX	bgrXXXXrb XgrXXXXrb gXrXXXXrb bXrXXXXrb rXXXXXXrb bgrXXXXXr bgrXXXXrX XXrXXXXrb XrXXXXXrb	XgrXXXXXr gXrXXXXrX gXrXXXXXr bXrXXXXrX bXrXXXXXr rXXXXXXXr rXXXXXXrX XgrXXXXXr gXrXXXXXr bXrXXXXXr rXXXXXXXr XgrXXXXrX gXrXXXXrX bXrXXXXrX rXXXXXXrX XXrXXXXXr XXrXXXXrX XrXXXXXXr XrXXXXXrX XXrXXXXrX XrXXXXXrX
XgrXXXXXr	bgrXXXXrb XgrXXXXrb	gXrXXXXrX gXrXXXXXr

	gXrXXXXrb bXrXXXXrb rXXXXXXrb bgrXXXXXr bgrXXXXrX XXrXXXXrb XrXXXXXrb XgrXXXXrX	bXrXXXXrX bXrXXXXXr rXXXXXXXr rXXXXXXrX XgrXXXXXr gXrXXXXXr bXrXXXXXr rXXXXXXXr XgrXXXXrX gXrXXXXrX bXrXXXXrX rXXXXXXrX XXrXXXXXr XXrXXXXrX XrXXXXXXr XrXXXXXrX XXrXXXXrX XrXXXXXrX XXrXXXXXr XrXXXXXXr
gXrXXXXrX	bgrXXXXrb XgrXXXXrb gXrXXXXrb bXrXXXXrb rXXXXXXrb bgrXXXXXr bgrXXXXrX XXrXXXXrb XrXXXXXrb XgrXXXXrX XgrXXXXXr	gXrXXXXrX gXrXXXXXr bXrXXXXrX bXrXXXXXr rXXXXXXXr rXXXXXXrX XgrXXXXXr gXrXXXXXr bXrXXXXXr rXXXXXXXr XgrXXXXrX gXrXXXXrX bXrXXXXrX rXXXXXXrX XXrXXXXXr XXrXXXXrX XrXXXXXXr XrXXXXXrX XXrXXXXrX XrXXXXXrX XXrXXXXXr XrXXXXXXr

El proceso termina cuando gXrXXXXrX no puede expandirse. Es un estado objetivo.